

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И. И. Ползунова»

Факультет информационных технологий
Кафедра прикладной математики

Отчет защищен с оценкой _____

Преподаватель _____
(подпись)

«___» _____ 2026 г.

Отчет
По лабораторной работе №7

«Файлы. Творческие задачи»

по дисциплине «Программирование – 2 семестр»

Студент группы ПИ-54 Дворников М. С.

Преподаватель доцент, к.т.н. Егорова Е.В.

Барнаул 2026

Задание. Написать игру.

Я выбрал игру «змейка»

Текст программы:

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h> // Стандартный вход/выход
#include <locale.h> // Вывод на русском языке
#include <conio.h> // Задержка экрана

#include <string.h> // Работа со строками
#include <math.h> // Математические функции

#include <stdlib.h> // Создание динамических массивов, использования рандома и преобразования строк
#include <windows.h> // Очистка консоли и установка кодировки
#include <time.h> // Для обновления рандома

#define SETIO SetConsoleCP(1251); SetConsoleOutputCP(1251); // Ввод и вывод на русском
#define UTF SetConsoleCP(65001); SetConsoleOutputCP(65001); // Ввод и вывод на кодировке UTF

#define FPS 120 // Кадры в секунду
#define SPEED (FPS / 12) // Скорость змейки
#define DRAWS "a X@o" // Графика для рисования

#define SIZE 25 // Размер поля

// Предметы на поле
#define APPLE -1 // Яблоко
#define GRASS 0 // Тут пусто
#define WALL 1 // Стенка
#define SNAKE 2 // А-А-А, ЗМЕЯ!!!

#define START 2 // Начальная длина тела змейки

// Направления змейки
#define UP 1 // Вверх
#define RIGHT 2 // Направо
#define DOWN 3 // Вниз
#define LEFT 4 // Влево

// Структура змейки
typedef struct {
    int body_x[SIZE * SIZE]; // Координаты частей тела по x
    int body_y[SIZE * SIZE]; // Координаты частей тела по y
    int head[2]; // Координаты головы (x, y)
    int size; // Текущая длина (только тело)
    int direct; // Текущее направление
} Snake;

// Структура для записи
typedef struct {
    char name[16];
    int score;
} Record;

// Создать яблоко на поле
int make_apple(int field[SIZE][SIZE]);

// Отрисовать кадр
void draw_screen(int field[SIZE][SIZE], Snake snake, int debug=0);

// Передвинуть змейку
void move_snake(int field[SIZE][SIZE], Snake *snake);
```

```

// Записать счёт в файл
void write_score(int score, char name[16]);

// Посмотреть таблицу лидеров
void leaderboard();

int main() {
    SETIO;
    system("color F0");
    srand(time(NULL)); // Обновляем рандом
    // Получим имя пользователя
    printf("Введите ваше имя (не более 15 символов): ");
    char name[16];
    int name_len = 0;
    while (name_len == 0) {
        fgets(name, 16, stdin);
        name_len = strlen(name);
        if (name[name_len - 1] == '\n') name[--name_len] = '\0';
    }
    printf("Здравствуй, %s\n", name);
    printf("Нажми любую кнопку, чтобы продолжить");
    int user_answer = _getch();
    do {
        int field[SIZE][SIZE]; // Создаём поле
        for (int i = 0; i < SIZE; i++)
            for (int j = 0; j < SIZE; j++)
                if (i == 0 || i == SIZE - 1 || j == 0 || j == SIZE - 1)
                    field[i][j] = WALL; // Стенки по краям
                else field[i][j] = GRASS;

        Snake snake; // Создаём змейку
        snake.direct = DOWN;
        snake.size = START;
        snake.head[0] = snake.head[1] = SIZE / 2;
        field[SIZE / 2][SIZE / 2] = SNAKE;
        for (int i = 0; i < snake.size; i++) {
            snake.body_x[i] = SIZE / 2;
            snake.body_y[i] = SIZE / 2 - 1 - i;
            field[snake.body_y[i]][snake.body_x[i]] = SNAKE;
        }

        make_apple(field); // Добавляем яблоко

        // В какое направление пользователь хочет повернуть змейку
        int new_dir = DOWN;
        int tick_counter = 1; // Счётчик кадров
        int run = 1; // Игра работает
        while (run) {
            // Отрисовываем кадры
            if (tick_counter == 1)
                draw_screen(field, snake);

            // Обрабатываем нажатия клавиш
            while (_kbhit()) { // Клавиша была нажата
                int key = _getch(); // Код нажатой клавиши
                if (key == 224) { // Стрелка даёт два кода - проверяем первый
                    key = _getch(); // Настоящий код стрелочки
                    if (key == 72 && snake.direct != DOWN) new_dir = UP;
                    else if (key == 75 && snake.direct != RIGHT) new_dir = LEFT;
                    else if (key == 77 && snake.direct != LEFT) new_dir = RIGHT;
                    else if (key == 80 && snake.direct != UP) new_dir = DOWN;
                }
                else {
                    if ((key == 'w' || key == 'W' || key == 'ц' || key == 'Ц')
                        && snake.direct != DOWN) new_dir = UP;
                }
            }
        }
    }
}

```

```

        else if ((key == 'd' || key == 'D' || key == 'Б' || key ==
'B')
                && snake.direct != LEFT) new_dir = RIGHT;
        else if ((key == 's' || key == 'S' || key == 'Ы' || key ==
'Ы')
                && snake.direct != UP) new_dir = DOWN;
        else if ((key == 'a' || key == 'A' || key == 'Ф' || key ==
'Ф')
                && snake.direct != RIGHT) new_dir = LEFT;
    }
}
// Обрабатываем информацию раз в определённое время
if (tick_counter * SPEED >= FPS) {
    snake.direct = new_dir;
    // Обрабатываем информацию
    int next_cell; // Что перед змейкой
    if (snake.direct == UP)
        next_cell = field[snake.head[1] - 1][snake.head[0]];
    else if (snake.direct == RIGHT)
        next_cell = field[snake.head[1]][snake.head[0] + 1];
    else if (snake.direct == DOWN)
        next_cell = field[snake.head[1] + 1][snake.head[0]];
    else if (snake.direct == LEFT)
        next_cell = field[snake.head[1]][snake.head[0] - 1];

    if (next_cell == GRASS) // Спереди ничего нет
        move_snake(field, &snake);
    else if (next_cell == APPLE) { // Спереди яблоко
        int old_end[2]; // Старые координаты конца
        old_end[0] = snake.body_x[snake.size - 1];
        old_end[1] = snake.body_y[snake.size - 1];
        move_snake(field, &snake); // Двигаем змейку
        snake.size++; // Добавим новый сегмент
        snake.body_x[snake.size - 1] = old_end[0];
        snake.body_y[snake.size - 1] = old_end[1];
        make_apple(field); // Добавим новое яблоко
    }
    else run = 0;
    tick_counter = 0;
}
tick_counter++;
Sleep(1000 / FPS); // Для FPS числа тиков в секунду
}
system("cls");
printf("Игра окончена. Счёт: %d\n", snake.size - START);
printf("Нажмите ESC, чтобы выйти.\n");
printf("Нажмите Enter, чтобы сохранить свой результат.\n");
printf("Нажмите пробел, чтобы посмотреть таблицу лидеров.\n");
printf("Нажмите любую другую кнопку, чтобы начать снова");
user_answer = _getch();
if (user_answer == 13) {
    system("cls");
    write_score(snake.size - START, name);
    printf("Нажмите ESC, чтобы выйти.\n");
    printf("Нажмите пробел, чтобы посмотреть таблицу лидеров.\n");
    printf("Нажмите любую другую кнопку, чтобы начать снова");
    user_answer = _getch();
}
if (user_answer == ' ') {
    system("cls");
    leaderboard();
    printf("\nНажмите ESC, чтобы выйти.\n");
    printf("Нажмите любую другую кнопку, чтобы начать снова");
    user_answer = _getch();
}
} while (user_answer != 27);

```

```

}

// Создать яблоко на поле
int make_apple(int field[SIZE][SIZE]) {
    for (int cur_try = 0; cur_try < 10; cur_try++) {
        // Пытаемся случайно попасть на пустую клетку 10 раз
        int x = rand() % SIZE;
        int y = rand() % SIZE;
        if (!field[y][x]) {
            field[y][x] = APPLE; // Ставим яблоко на пустое место
            return 1;
        }
    }
    // Если первые 10 раз не получилось поставить яблоко,
    // ставим в первую попавшуюся клетку
    for (int y = 0; y < SIZE; y++)
        for (int x = 0; x < SIZE; x++)
            if (!field[y][x]) {
                field[y][x] = APPLE; // Ставим яблоко на пустое место
                return 2;
            }
    return 0; // Если не поставили яблоко, то возвращаем 0
}

// Отрисовать кадр
void draw_screen(int field[SIZE][SIZE], Snake snake, int debug) {
    system("cls");
    if (debug) {
        printf("size = %d, direct = %d\n", snake.size, snake.direct);
        printf("head: %d, %d\n", snake.head[0], snake.head[1]);
        for (int i = 0; i < snake.size; i++)
            printf("body %d: %d, %d\n", i + 1, snake.body_x[i], snake.body_y[i]);
    }
    for (int y = 0; y < SIZE; y++) {
        printf("\t");
        for (int x = 0; x < SIZE; x++)
            if (field[y][x] < SNAKE) // Если это не змейка
                printf("%c", DRAWS[field[y][x] + 1]);
            else // Разделяем голову и тело
                if (snake.head[0] == x && snake.head[1] == y)
                    printf("%c", DRAWS[3]); // Голова
                else printf("%c", DRAWS[4]); // Тело
        printf("\n");
    }
    printf("\n\tСчёт: %d\n", snake.size - START);
}

// Передвинуть змейку
void move_snake(int field[SIZE][SIZE], Snake *snake) {
    // Удалим с поля кончик змеи
    int ss = (*snake).size; // Сократим путь до длины змейки
    field[(*snake).body_y[ss - 1]][(*snake).body_x[ss - 1]] = GRASS;
    // Сначала сдвинем всё туловище
    for (int i = ss - 1; i > 0; i--) {
        (*snake).body_x[i] = (*snake).body_x[i - 1];
        (*snake).body_y[i] = (*snake).body_y[i - 1];
    }
    (*snake).body_x[0] = (*snake).head[0];
    (*snake).body_y[0] = (*snake).head[1];
    // Потом двигаем голову
    if ((*snake).direct == UP) (*snake).head[1]--;
    else if ((*snake).direct == RIGHT) (*snake).head[0]++;
    else if ((*snake).direct == DOWN) (*snake).head[1]++;
    else if ((*snake).direct == LEFT) (*snake).head[0]--;
    // Компенсируем удалённый кончик на поле головой
    field[(*snake).head[1]][(*snake).head[0]] = SNAKE;
}

```

```

}

// Записать счёт в файл
void write_score(int score, char name[16]) {
    // Открываем файл для добавления (если файла нет - создаётся)
    FILE* file = fopen("scores.txt", "a");

    if (file == NULL) {
        printf("Ошибка: не удалось открыть файл для записи!\n");
        return;
    }

    // Записываем имя и счёт в файл
    fprintf(file, "%s %d\n", name, score);

    // Закрываем файл
    fclose(file);
}

// Посмотреть таблицу лидеров
void leaderboard() {
    FILE* file = fopen("scores.txt", "r");
    if (file == NULL) {
        printf("Таблица лидеров пока пуста!\n");
        return;
    }

    Record records[100];
    int count = 0;
    // Читаем все записи из файла
    while (fscanf(file, "%s %d", records[count].name, &records[count].score) == 2 ) {
        count++;
        if (count >= 100) break;
    }
    fclose(file);

    if (count == 0) {
        printf("Таблица лидеров пока пуста!\n");
        return;
    }

    // Сортируем по убыванию счёта
    for (int i = 0; i < count - 1; i++)
        for (int j = 0; j < count - i - 1; j++)
            if (records[j].score < records[j + 1].score) {
                Record temp = records[j];
                records[j] = records[j + 1];
                records[j + 1] = temp;
            }

    // Выводим таблицу
    printf("ТАБЛИЦА ЛИДЕРОВ\n");
    for (int i = 0; i < count && i < 10; i++) // Топ-10
        printf("%d место:\t%s: %d\n", i + 1, records[i].name, records[i].score);
}

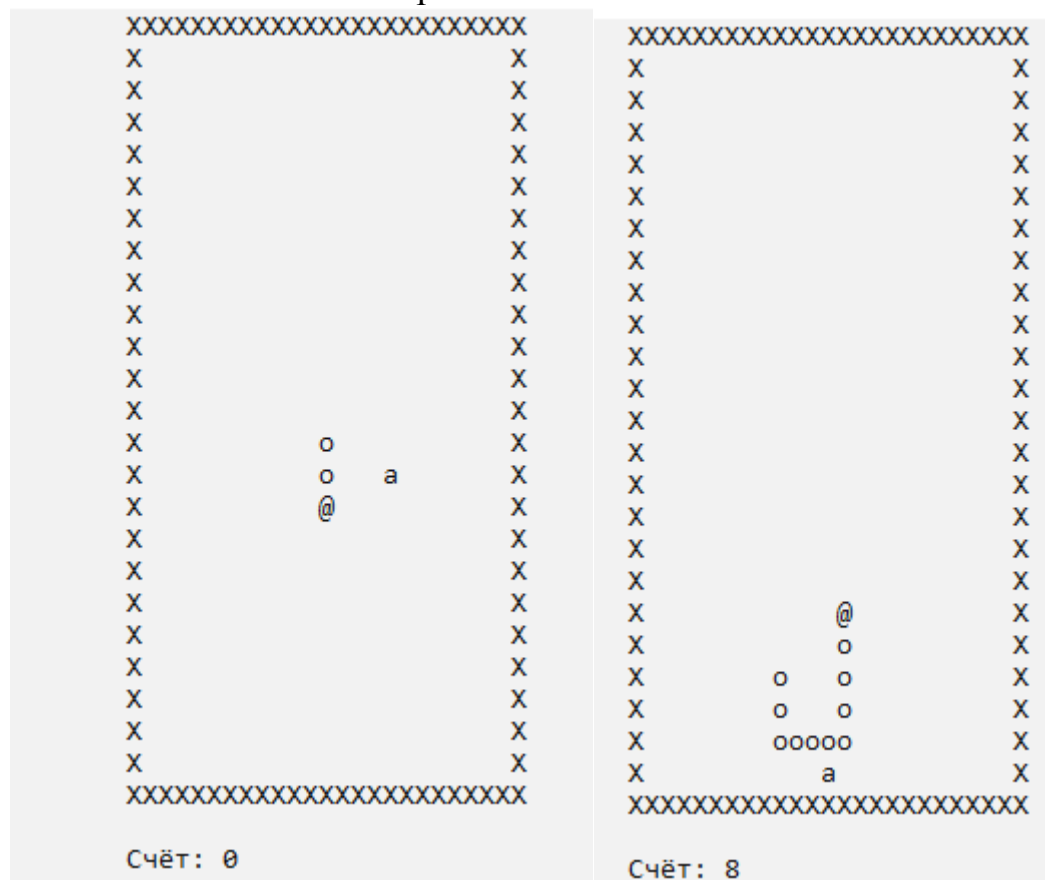
```

Тесты программы:

Игрока встречает меню выбора имени:

Введите ваше имя (не более 15 символов): Марк
Здравствуй, Марк
Нажми любую кнопку, чтобы продолжить

Затем начинается сама игра



После проигрыша программа показывает следующее меню:

Игра окончена. Счёт: 0
Нажмите ESC, чтобы выйти.
Нажмите Enter, чтобы сохранить свой результат.
Нажмите пробел, чтобы посмотреть таблицу лидеров.
Нажмите любую другую кнопку, чтобы начать снова

ESC – завершает программу

Нажатие на Enter:

Нажмите ESC, чтобы выйти.
Нажмите пробел, чтобы посмотреть таблицу лидеров.
Нажмите любую другую кнопку, чтобы начать снова

Нажатие на пробел:

ТАБЛИЦА ЛИДЕРОВ
1 место: Маркуша: 5
2 место: Маркуша: 2
Нажмите ESC, чтобы выйти.
Нажмите любую другую кнопку, чтобы начать снова